

Module 5: Embedded Networking

COMPLETE LAB SHEET IN BEFORE YOU BEGIN THE NEXT LAB

Write your name legibly here:

Grade:

Learning Objectives:

- Configuring a USART with interrupts.
- Introduce networking interfacing using UDP.
- Introduce embedded networking devices.
- Controlling an external WiFi device using a USART.
- Develop a first, completely *event-driven* program.



Note that Xilinx uses the shortened term UART, rather than USART, in its coding practices to mean the same thing – they are used here interchangeably.

Step 1. Introduction. Read and make notes on the following:

- **Module 5:** M5.0 Introduction (video)
- **Zynq Reference Manual (UG585):** Chapter 19: USART
- **WiFly Module Data Sheet:** Note the pin locations.
- **WiFly Command Reference Manual:** Sections 3.1, 5.6, 5.11, 6, 8, and 11. Determine how to enter the wireless module’s “command” mode. Note down the Wi-Fly modules default UART interface configuration.
- Search online to discover the difference between “big-endian” and “little-endian”.

Step 2. Obtain your unique class identifier from the class roster (a number in the range 0-29). Create a new hardware project – module5 -- with just a Zynq processor and AXI GPIO ports connected to the leds, buttons, and switches. Make sure you add the wrapper and generate the bitstream. Start the SDK, use the default *helloworld.c* program, and make sure it prints “Hello World” on the screen.

Import your **led**, **io** and **gic** modules from Module 4; you do ***not*** need the **ttc** module. Using the functions in Table 5.1 and the **gic** module, write an *interrupt* handler for UART1 that echo’s each character typed on the keyboard back onto the screen. Your program should exit when button 3 is pressed, properly disconnecting the UART1 from the GIC, closing down the GIC, and returning the hardware into a state where it can be reused. Make sure that you turn off the board and re-program the FPGA after every run until you are able to reliably receive interrupts and exit the program cleaning up the hardware properly. With random typing (including <return>’s) your program output should look something like:

```
[hello]
adsf fgdf
asdfa dfa sdfasdf
```

```
aasdfas df
asdfasdf
[done]
```

Your main program should do nothing and look like this:

```
int main(void) {
    /* do some initialization here */
    printf("[hello]\n");
    while(!done) /* do nothing waiting for interrupts */
        sleep(1); /* included from <unistd.h> */
    printf("[done]\n");
    sleep(1);
    /* do some cleanup here */
}
```

Note that unlike AXI GPIO's, UARTs connected to the processor provide a predefined interrupt handler **XUartPs_InterruptHandler** that must be connected to the GIC; this handler eventually invokes a user defined handler that has the general form:

```
void handler(void *CallBackRef, u32 Event, unsigned int EventData);
```

where *CallBackRef* is a void pointer to an instance of a UART, just like AXI GPIO handlers.

To check if a *receive data* interrupt has been triggered (as opposed to some other interrupt) your *handler* should check the event type i.e.:

```
if (Event == XUARTPS_EVENT_RECV_DATA) {
    // do something appropriate
}
```

The following constants will be helpful:

```
XPAR_PS7_UART_1_DEVICE_ID
XPAR_XUARTPS_1_INTR
```

(3 points) TA Signature:

Table 5.1: Uart Functions (xuartps.h)

Function	Description
XUartPs_LookupConfig	Looks up the device configuration based on the unique device ID in a table that contains info for each device in the system
XUartPs_CfgInitialize	Initializes a specific XUartPs instance in a default configuration, such that it is ready to be used
XUartPs_SetBaudRate	Sets the baud rate for the device
XUartPs_SetFifoThreshold	Sets the FIFO trigger threshold
XUartPs_SetInterruptMask	Sets the interrupt mask
XUartPs_SetHandler	Sets the handler that will be called when an event (interrupt) occurs that needs application's attention
XUartPs_DisableUart	Disable a Uart
XUartPs_Send	Sends the specified buffer using the device in either polled or interrupt driven mode
XUartPs_Recv	Attempts to receive a specified number of bytes of data from the device and store it into the specified buffer

Step 3. Open up the Zynq processor and enable UART0; connect it to receive (Rx) on MIO 10 and transmit (Tx) on MIO 11 – (MIO 10/11 is connected to PMOD JF pins 2/3). Note that UART1 remains connected to the USB.

When UART0 is enabled, the standard input base address and standard output base address in the file “xparameter.h” file is changed from 0xE0001000 (USART1) to 0xE0000000 (USART0). This will cause the *printf()* function stop printing to the terminal emulator. Edit the “xparameters.h” file so that the standard input/output base addresses are E0001000 again, this function works again. Verify your program operates as it did previously but with the new hardware setup.

Configure UART0 to operate at the default baud rate of the Wifi module determined in Step 1. Wire the USART0 Rx and Tx PMOD pins together to provide a loopback from the output to input (shown right); Consequently, any value transmitted by USART0 will be show up back at its own input.



Next, implement an interrupt handler for UART0 that forwards any characters received *by* UART0 *to* UART1. The idea is that any data received *from* the Wi-Fly module should be sent via UART1 *to* the terminal window on your development workstation.

Modify the interrupt handler for UART1 such that it forwards any characters received *by* UART1 *to* UART0. The idea is that any data received *from* the terminal window on your development workstation should be sent over UART0 *to* the Wi-Fly module.

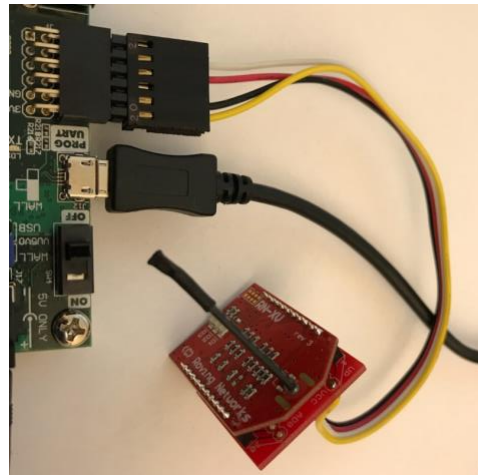
Check that anything you type is echo'ed back via the loopback connection. Note that each character you type on the keyboard should be echo'ed only once on the terminal: when it arrives back at UART1 by virtue of the loopback connection at UART0.

(3 points) TA Signature:

Step 4. Remove your loopback connection and connect your WiFi module to the board on **PMOD JF** using the supplied harness. Note that there are several alternative wiring harnesses that connect to pins {2,3,5,6} respectively:

{White, Red, Black, Yellow} -- (shown right)
{Orange, Yellow, Green, Red}
{Grey, White, Black, Purple}
{Orange, Red, Beige, Yellow}

Be careful to ensure that you connect the device correctly to the PMOD connector.



Demonstrate that you can communicate with the Wi-Fly module by entering the Wi-Fly module's command mode. Once in command mode issue the command "**get wlan**" to view the current WLAN settings, and make sure they are printed onto the terminal.

(2 points) TA Signature:

Step 5. Perform a factory reset on your device using the command “factory RESET”. Review your notes on sections 5.6, 5.11, 6, 8, and 11 of the WiFi module’s user manual, and recall that the table under “**set ip protocol** <value>” presents the configuration in terms of *bit position*, not <value>.

M210 provides an isolated open (no password) HIDDEN wireless network using a wireless access point configured with:

SSID: engs62

The access point provides wireless modules with DHCP-based network configuration, issuing your WiFi module with an IP address in the range **192.168.1.100** to **192.168.1.200**. The WiFi module may then communicate with a “cloud server”, located elsewhere on the internet, whose IP address is **129.170.136.11**. You can view the screen on the remote server by logging into the Dartmouth VPN and then visiting: <https://engs62.thayer.dartmouth.edu> in a browser. This server provides a UDP service on port **8880**. The behavior of this service will be described later; for now, simply configure the Wi-Fly module to associate with the ENGS62 wireless network and, when in data mode, to send data via UDP to the remote host 129.170.136.11 on port 8880. To do this, at a minimum, you must:

- Issue the character sequence to enter the Wi-Fly’s command mode.
- Set the join mode to auto-connect to the stored network.
- Set the wireless authentication to none
- Set the wireless SSID to “engs62”
- Set the IP configuration to use DHCP from the access point.
- Set the IP remote host to 129.170.136.11
- Set the IP remote port number to 8880
- Set the IP protocol to UDP
- Save the configuration to the module’s non-volatile memory.

NOTE: once the configuration is saved to the module’s non-volatile memory you do not need to re-enter it on each power-up.

Demonstrate that your wireless module automatically associates with the ENGS62 network on power up. You can use the commands “**get ip**” and “**get wlan**” to show your WiFi modules current settings.

(3 points) TA Signature:

Step 6. Define 3 constants at the top of the file containing your main program:

```
#define CONFIGURE 0
#define PING 1
#define UPDATE 2
```

Modify your program to provide 3 modes of operation: CONFIGURE, PING, and UPDATE. Each mode is entered when an associated button is pressed (0,1,2 respectively) causing the corresponding led to come on alone; control remains in the given mode until another button is pressed.

The CONFIGURE mode should operate as in Step 1, allowing entry to the wifi command mode, sending anything typed to the *wifi* module, and echo'ing anything received from the wifi module. The other modes should simply print a message on the screen, [PING] or [UPDATE] respectively, when entering the mode. Switching between modes and typing random input while in CONFIGURE mode should present output something like this:

[hello]	← button 0 pressed here
< allows entry to wifi cmd mode >	
:	← button 1 pressed here
[PING]	← button 1 pressed again
[PING]	← button 1 pressed again
[PING]	← button 2 pressed here
[UPDATE]	← button 2 pressed again
[UPDATE]	← button 0 pressed here
< allows entry to wifi cmd mode >	
:	← button 1 pressed here
[PING]	← button 1 pressed again
[PING]	← button 2 pressed here
[UPDATE]	← button 2 pressed again
[UPDATE]	← button 0 pressed here
<allows entry to wifi cmd mode >	
:	← button 3 pressed here
[done]	← program exists here

(1 points) TA Signature:

Step 7. Adapt your program to refine your **PING** mode as follows:

- Use the message type – PING -- and structure definition to construct a PING message. Use the unique identifier assigned to you at the start of this module for the id field (30 id's are available; id is a number in the range 0-29):

```
#define PING 1    /* defined in step 6 */

typedef struct {
    int type;    /* must be assigned to PING */
    int id;      /* must be assigned to your id */
} ping_t;
```

- Decode and display the server's response (the server will echo all correctly-formatted ping packets back to the sender).

Demonstrate that your wireless module can ping the echo server and receives the expected response.

(3 points) TA Signature:

Step 8. Adapt your program to include an UPDATE mode that operates as follows:

- Use the following message type (UPDATE) and structure definition to construct an UPDATE message. Use the unique identifier assigned to you at the start of this module for the id field.

```
#define UPDATE 2 /* defined in step 6 */

typedef struct {
    int type; /* must be assigned to UPDATE */
    int id;   /* must be assigned to your id */
    int value; /* must be assigned to some value */
} update_request_t;
```

- Send the update packet to the server where the *value* is entered on the keyboard. Verify that you see your update packet in the server console's diagnostic output at the front of the class.
- Decode and display the server's response. The format of the response message is as follows, where type is the value UPDATE, average is the average value from all the class wireless devices, and values[30] is an array of the last update value from each wireless device, indexed by id:

```
typedef struct {  
    int type;  
    int id;  
    int average;  
    int values[30];  
} update_response_t;
```

Demonstrate that your wireless module sends updates to the echo server and receives the expected response.

(3 points) TA Signature:

Step 9. Integrate the servo and potentiometer into your solution such that the value sent to the server by the UPDATE mode is a percentage reading of your potentiometer (where 0v=0%; 1.0v=100%). The update response should be used to move the servo to an appropriate position.

(2 points) TA Signature: